

D. Summit or Stuck?



Time Constraints

- **java** : 3.0 Seconds



Memory Constraints

- **java** : 256MB

A rectangular climbing wall is represented by an integer height-map H of size $R \times C$, where $H[i][j]$ gives the height (in metres) of hold (i, j) . Row indices run $0 \dots R-1$ from top to bottom, and column indices run $0 \dots C-1$ from left to right.

Movement rule

From any hold (i, j) the climber may move once to any of the eight surrounding cells (up, down, left, right, or any diagonal) only if the destination is strictly higher:

$$H[i'][j'] > H[i][j]$$

Task

Simulate Steepest-Ascent Rock Climbing (always choose the neighbour with the largest height; break ties arbitrarily) starting from a given starting hold (r_s, c_s) .

Your program must output the word GLOBAL MAX if the final hold is a global maximum of the grid, otherwise LOCAL MAX.

The algorithm terminates when no neighbouring hold is strictly higher.

Input

- $1 \leq R, C \leq 100$
- Matrix of size $R \times C$
- r_s, c_s
- All heights $H[i][j]$ are integers (positive, zero, or negative).

Output

- print either GLOBAL MAX or LOCAL MAX (depending on whether the terminal hold is the highest in the entire grid).

****Example 1 ****

Sample Input

```
7 6
3 7 5 4 3 0
4 8 10 6 2 1
2 9 12 7 3 2
1 5 8 5 4 3
0 3 6 4 1 0
0 2 4 2 0 0
0 0 0 0 0 0
3 1
```

Sample Output

GLOBAL MAX

Explanation

Starting at $(3, 1)$ (height 5), the climber's steepest admissible neighbour is $(2, 2)$ (height 12). No neighbour of $(2, 2)$ is higher, so the algorithm stops. Height 12 is the largest value in the grid, hence the result is GLOBAL MAX.

Example 2